

CHAPTER 12

12 - Prisma 관계 (판매자 1:N · 분류 M:N)

Node.js · Express · NestJS 강의 · 2026

11장의 상품은 `categoryId` 라는 숫자 칸만 있었지, 진짜 분류와 연결돼 있지는 않았다. 이 장은 상품을 두 가지 관계로 잇는다.

- 판매자(User) 1:N 상품 — 한 사람이 여러 상품을 판다. 상품에 `sellerId` 외래키가 생긴다.
- 분류(Category) M:N 상품 — 한 상품이 여러 분류에 속한다. 11장의 단일 `categoryId` 를 다대다로 승격한다.

User를 새로 추가하지만, 아직 "데이터"일 뿐이다(비밀번호·로그인 없음). 13장에서 이 User에 인증을 얹는다. 그래서 지금 상품의 `sellerId` 는 요청 본문으로 받는다 — 13장에서 "로그인한 사용자(토큰)"로 바뀐다.

이 장에서 달라진 것 (← 11장 대비)

구분	파일 / 내용
신규	<code>src/users/</code> (User CRUD 모듈)
수정	<code>prisma/schema.prisma</code> — User 모델 + <code>Product.sellerId(1:N)</code> + <code>Product.categories(M:N)</code> · <code>products.service</code> — <code>connect</code> / <code>set</code> / <code>include</code> 로 관계 처리 · <code>products/dto</code> — <code>sellerId</code> · <code>categoryIds</code> 추가, <code>UpdateDto</code> 를 <code>PartialType</code> 로
의존성	<code>+@nestjs/mapped-types</code>
엔드포인트	<code>+POST/GET /users</code> · <code>GET /users/:id</code> · <code>GET /categories/:id</code> 는 빠짐(분류 검증이 상품 생성 안쪽으로 들어가 단건 조회 라우트가 불필요해짐) · 상품 응답에 <code>seller</code> · <code>categories</code> 포함, <code>?categoryId=</code> 필터

0. 사전 준비 — PostgreSQL

PostgreSQL이 떠 있어야 한다(11장에서 접속한 그 서버). DBeaver에서 연결이 열리면 준비된 상태다. 접속·확인 방법은 11장 0번 참고.

1. 패키지 설치

```
npm i @nestjs/mapped-types@2
```

```
# UpdateDto를 PartialType으로 줄이기 위해
```

10장에서 `nest g resource` 를 썼다면 이미 설치돼 있던 패키지다. shop 프로젝트에는 없으니 여기서 넣는다.

2. 모듈 생성

```
nest g module users
nest g service users --no-spec
nest g controller users --no-spec
```

3. 스키마 — 관계 걸기 + 마이그레이션

prisma/schema.prisma 에 User 모델을 더하고, Product의 categoryId (숫자 칸)를 지우고 진짜 관계로 바꾼다.

```
model User {
  id          Int          @id @default(autoincrement())
  email       String       @unique
  name        String
  products    Product[]    // 이 사용자가 판매자로 등록한 상품들 (1:N)
  createdAt   DateTime     @default(now())
}

model Category {
  id          Int          @id @default(autoincrement())
  name        String       @unique
  products    Product[]    // M:N - Prisma가 중간 테이블을 자동으로 만든다
}

model Product {
  id          Int          @id @default(autoincrement())
  name        String
  description  String
  price       Int
  stock       Int          @default(0)
  seller      User         @relation(fields: [sellerId], references: [id])
  sellerId    Int          // 11장의 categoryId처럼 보이지만, 이건 진짜 외래키다
  categories  Category[]   // M:N - 단일 categoryId는 삭제
  createdAt   DateTime     @default(now())
}
```

```
npm run prisma migrate dev --name add-relations
```

11장에서 넣어 본 상품이 DB에 남아 있으면, 필수 컬럼 sellerId 를 더할 수 없어 Prisma가 리셋을 물어볼 수 있다. 개발 DB이므로 동의하면 된다 — 데이터만 비워지고 마이그레이션 이력은 유지된다.

4. 코드 작성 순서

순 서	파일	내용
1	<code>src/users/dto/create-user.dto.ts</code>	<code>email</code> · <code>name</code> 검증
2	<code>src/users/users.service.ts</code>	생성·목록·단건(<code>findOne</code> 없으면 404)
3	<code>src/users/users.controller.ts</code> · <code>users.module.ts</code>	<code>POST/GET /users</code> · <code>GET /users/:id</code> · <code>UserService</code> 를 <code>exports</code>
4	<code>src/categories/categories.service.ts</code>	<code>ensureAllExist(ids)</code> 추가 — 없는 분류가 섞이면 400
5	<code>src/products/dto/*.ts</code>	<code>sellerId</code> · <code>categoryIds</code> 추가 · <code>UpdateProductDto</code> extends <code>PartialType(CreateProductDto)</code>
6	<code>src/products/products.service.ts</code>	<code>connect</code> / <code>set</code> 으로 관계 연결, <code>include</code> 로 <code>seller-categories</code> 동봉, <code>?categoryId=</code> 필터
7	<code>src/products/products.module.ts</code>	<code>UsersModule</code> 을 <code>imports</code>
8	<code>src/categories/categories.controller.ts</code>	<code>GET /categories/:id</code> 라우트 제거(검증이 상품 생성 안쪽으로 들어갔다)

각 파일의 코드와 설명은 해설서 12장에 있다.

5. 동작 테스트 (curl)

`BASE=http://localhost:3000`

1) 판매자(User) 생성

```
curl -s -X POST $BASE/users -H "Content-Type: application/json" \
  -d '{"email":"seller@example.com","name":"판매자"}'
```

2) 분류 두 개

```
curl -s -X POST $BASE/categories -H "Content-Type: application/json" -d '{"name":"전자기"}'
```

```
curl -s -X POST $BASE/categories -H "Content-Type: application/json" -d '{"name":"음향기"}'
```

3) 상품 등록 - sellerId + categoryIds. 응답에 seller.categories가 함께 온다.

```
curl -s -X POST $BASE/products -H "Content-Type: application/json" \
  -d '{"name":"무선 이어폰","description":"노이즈 캔슬링","price":89000,"stock":10,"sellerId":1,"categoryIds":[1,2]}'
```

4) 없는 판매자 → 404 / 없는 분류 → 400

```
curl -i -X POST $BASE/products -H "Content-Type: application/json" \
  -d '{"name":"x","description":"x","price":1,"stock":1,"sellerId":999,"categoryIds":[1]}'
```

404

5) M:N 필터 - 분류 2를 포함한 상품

```
curl -s "$BASE/products?categoryId=2"
```

6) 분류 연결 통째로 교체 (set)

```
curl -s -X PATCH $BASE/products/1 -H "Content-Type: application/json" -d '{"categoryIds":[1]}'
```

핵심 개념

1:N — 외래키와 include

`Product.sellerId` 가 `User.id` 를 가리키는 외래키다(11장의 스칼라 `categoryId` 와 달리 `@relation` 으로 묶였다). 조회할 때 `include` 로 판매자 정보를 함께 끌어온다.

```
this.prisma.product.findUnique({
  where: { id },
  include: { seller: { select: { id: true, name: true } } },
});
```

M:N — connect 와 set

상품 하나가 여러 분류에 속한다. `Product.categories Category[]` / `Category.products Product[]` 양쪽에 배열을 두면 Prisma가 중간 테이블을 자동으로 만든다.

- 새로 연결: `categories: { connect: [{ id: 1 }, { id: 2 }] }`
- 통째로 교체: `categories: { set: [{ id: 1 }] }`
- 그 분류를 가진 상품 찾기: `where: { categories: { some: { id } } }`

PartialType — 반복 줄이기

11장의 `UpdateProductDto` 는 모든 필드를 `@IsOptional` 로 다시 적었다. `@nestjs/mapped-types` 의 `PartialType` 이 그 일을 대신한다 — 10장 posts의 `UpdatePostDto` 에서 이미 썼던 도구다.

```
export class UpdateProductDto extends PartialType(CreateProductDto) {}
```

핵심 파일

파일	역할
<code>prisma/schema.prisma</code>	User 1:N Product, Product M:N Category
<code>src/products/products.service.ts</code>	<code>connect</code> / <code>set</code> 으로 관계 연결, <code>include</code> 로 조회
<code>src/users/users.service.ts</code>	판매자 저장·조회(존재 확인용 <code>findOne</code>)
<code>src/categories/categories.service.ts</code>	<code>ensureAllExist</code> 로 분류 id 검증

따라잡기 — 완성본으로 실행 (보조 경로)

수업의 기본 경로는 위처럼 직접 만드는 것이다. 따라오다 놓쳤다면, 수업 저장소의 `12/` 폴더(완성 코드)로 이 장의 끝 상태에서 다시 시작할 수 있다.

```
cd 12
npm install
cp .env.example .env          # 사용자·비밀번호를 본인 PostgreSQL 값으로 수정
npx prisma migrate dev --name init  # 이 폴더 전용 DB(nest_shop_12) - 자동 생성이 안 되면
DBBeaver로 먼저 만든다
npm run start:dev
```

다음 장

- **13 - 인증:** User에 비밀번호·역할을 붙여 회원가입/로그인(JWT)을 만든다. 상품의 `sellerId` 는 더 이상 본문이 아니라 토큰에서 온다.